Maximizing Robustness to Vertex Failures in Size-Constrained Community Search

Yang Liu*, Hejiao Huang*, Lijun Chang†, Shengxin Liu*, and Zhaoquan Gu*
*Harbin Institute of Technology, Shenzhen, China
†The University of Sydney, Australia
23b951003@stu.hit.edu.cn, huanghejiao@hit.edu.cn, Lijun.Chang@sydney.edu.au,
sxliu@hit.edu.cn, guzhaoquan@hit.edu.cn

***Abstract*—Size-constrained community search (SCS) seeks a community containing a query vertex within a user-specified size range. Common SCS objectives emphasize cohesion, separation, or modularity, but do not directly limit reliance on a few vertices to maintain connectivity. Removing these vertices can fragment the community, disconnecting surviving members from the query vertex. To address this vulnerability, we formulate the *Size-Constrained Community Search Problem with Vertex Connectivity* (SVC), which maximizes vertex connectivity (VC) under query and size constraints. This objective provides an explicit robustness guarantee: a community with VC k remains connected after any set of fewer than k vertices is removed. We prove that SVC is NP-hard and develop exact branch-and-bound algorithms. Our approach exploits the observation that the difference between induced-subgraph size and VC never decreases as vertices are added. This property yields a pruning condition and guides a vertex-cut-based ordering that allows this condition to prune multiple branches earlier. Fast ordering construction, candidate reduction, and top-down search further accelerate the algorithms. Experiments on 12 real-world graphs show speedups of up to three orders of magnitude over the baseline. An Amazon co-purchasing case study further supports SVC's practical value. Our method achieves the highest F1-score across five size intervals. After two non-query vertices are randomly removed, it also yields the smallest mean absolute F1-score change in four of these intervals.**

***Index Terms*—Size-constrained community search, vertex connectivity, vertex-failure robustness, branch-and-bound.**

I. Introduction

Graphs are widely used to represent entities and their relationships in real-world applications [1], [2]. Many such networks, particularly social and biological ones, exhibit community structures, motivating extensive research on *Community Search* (CS) [3]–[10]. Given a graph G and a query vertex q, CS seeks a connected subgraph containing q that optimizes a specified community quality metric. CS supports diverse applications such as event organization [11] and social marketing [12].

Existing CS studies can be broadly classified into size-unconstrained and size-constrained settings. Size-unconstrained CS methods [7]–[10], [13] mainly optimize a community quality metric without imposing explicit bounds on community size. In contrast, size-constrained CS methods [14]–[21] require the returned community to satisfy application-specific size requirements. For example, in team formation, a lower size bound avoids groups too small to serve the intended purpose [22], while an upper bound keeps coordination

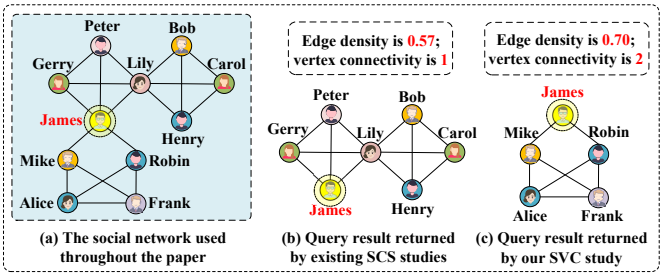


Fig. 1. Community search with $q =$ James and $[l, h] = [5, 7]$. (a) Input network. (b) Optimal community under the compared SCS objectives. (c) The optimal SVC community, remaining connected after any single vertex removal.

manageable [23]. Similarly, biological network analysis often focuses on functional modules within a desired size range [24], [25]. To meet such requirements, Yao and Chang [3] introduced the *Size-Constrained Community Search* (SCS) problem, which imposes both lower and upper bounds on community size. Given a size range $[l, h]$, SCS seeks a connected subgraph containing the query vertex whose size lies within $[l, h]$, while optimizing a specified community quality metric.

Existing Research. Existing SCS studies optimize different aspects of community quality to satisfy application-specific requirements. Minimum-degree-based methods [3], [26] maximize the minimum degree of the returned community to promote internal cohesiveness. Zhang et al. [5] studied a support-based formulation that maximizes the minimum vertex support. Here, a vertex's support is the maximum number of triangles containing any one of its incident edges. He et al. [4] proposed a conductance-based formulation, where lower conductance indicates relatively few connections to the rest of the graph compared with connections within the community. Recently, Kim et al. [27] introduced local sketch modularity (LSM), which combines internal and external connectivity with a size-aware penalty to mitigate the free-rider effect.

Limitations. The SCS objectives above characterize community quality but do not explicitly maximize tolerance to vertex failures. Consequently, optimizing them may still return a densely connected community that depends on a few critical members to remain connected. For example, in event organization, if these members become unavailable, remaining participants may lose all friendship paths to the organizer. Their absence can thus

fragment the community. Including participants who provide alternative paths can improve resilience, but their number is restricted by the size upper bound. The challenge is therefore to maximize resilience to member failures while satisfying the prescribed size constraints.

Example. Consider the social network in Figure 1(a), where $James$ plans to organize a party with 5 to 7 participants. The community in Figure 1(b) is optimal under the minimum-degree, vertex-support, conductance, and LSM objectives. However, $Lily$ is the only link between two circles of friends. If she cannot attend, the community splits, leaving some remaining participants without a friendship path to $James$ within the selected group. In contrast, the community in Figure 1(c) remains connected after any single vertex removal. Thus, if any participant other than $James$ becomes unavailable, all remaining participants stay connected to him. This illustrates why optimizing community quality under size constraints should also consider dependence on individual members.

We capture this dependence through *vertex connectivity* (VC), defined as the minimum number of vertices whose removal disconnects a graph or leaves at most one vertex. A community with VC k remains connected after any set of fewer than k vertices is removed. If the query vertex survives, all remaining members therefore stay connected to it. This robustness guarantee motivates VC-based graph analysis and community modeling [10], [28]–[35]. In particular, graph downsizing maximizes VC in an induced subgraph of a prescribed size [36]. However, it does not require the subgraph to contain a designated query vertex, as needed in our community search setting.

Our Research on SCS. We formulate the *Size-Constrained Community Search Problem with Vertex Connectivity* (SVC). Given a graph G, a query vertex q, and a size interval $[l, h]$, SVC seeks a connected induced subgraph containing q, with between l and h vertices, that maximizes vertex connectivity. It therefore selects a query community with the greatest tolerance to vertex failures attainable within the size budget.

We prove that SVC is NP-hard and develop exact algorithms within a *branch-and-bound* framework. We first design a heuristic to find a feasible community, whose VC provides an initial lower bound on the optimal value. We then explore combinations of candidate vertices through set-enumeration branching [37]. A key challenge is that an infeasible partial selection may become feasible as more vertices are added, so infeasibility alone does not justify pruning. To address this, we relate the size and VC of a partial selection to determine whether any extension within the size upper bound can improve the current best solution. Our key observation is that the difference between an induced subgraph's size and its VC never decreases as vertices are added. Together with the size upper bound, this property allows us to identify partial selections that cannot lead to a better community. We thus develop *Vertex Cut Last* (VCL), an ordering based on vertex-cut partitions that exposes such selections early and enables multiple branches to be pruned together. We further accelerate the search through fast VCL construction, candidate reduction, and a top-down strategy that explores tighter pruning thresholds first.

Our case study on the Amazon co-purchasing graph illustrates the practical value of SVC. Communities found by maximizing vertex connectivity match product categories more closely than those returned by the compared methods, achieving the highest F1-score across all five tested size intervals. Their category-recovery scores are also more stable after two random non-query vertex removals, with the smallest mean absolute change in four intervals. Together, these findings suggest that SVC can identify meaningful product groups and help preserve their usefulness when some members become unavailable.

Contributions. We make the following contributions:

- We formulate SVC, which maximizes vertex connectivity in a query-containing community under user-specified lower and upper size bounds, explicitly optimizing its tolerance to vertex failures. We prove that SVC is NP-hard. (Section II)
- We exploit the monotonicity of the difference between subgraph size and VC to derive a pruning condition under the size upper bound. Based on this condition, we design the VCL ordering to prune multiple branches together and incorporate it into exact branch-and-bound algorithms. (Section III)
- We further accelerate exact search through fast VCL construction to reduce ordering overhead, a reduction rule to eliminate unpromising candidates, and a top-down strategy that explores tighter pruning thresholds first. (Section IV)
- We evaluate our algorithms on 12 real-world graphs, achieving speedups of up to three orders of magnitude over the baseline. On Amazon, our method achieves the highest category-recovery F1-score in all five tested size intervals and the smallest mean absolute F1-score change in four intervals after two random non-query vertex removals. (Section V)

II. Preliminaries

We consider an unweighted and undirected graph $G = (V, E)$, where V is the vertex set, E is the edge set, $|V| = n$, and $|E| = m$. Given a vertex subset $S \subseteq V$, we use $G[S]$ to denote the subgraph induced by S, i.e., $G[S] = (S, \{(u, v) \in E \mid u, v \in S\})$. For a vertex v and a vertex set S, we define the neighbors of v in S as $N(v, S) = \{u \in S \mid (u, v) \in E\}$ and its degree in S as $d(v, S) = |N(v, S)|$. For an induced subgraph g of G, we denote its vertex set and edge set by $V(g)$ and $E(g)$, respectively.

A *vertex cut* of a graph $G = (V, E)$ is a vertex set $\Gamma \subseteq V$ such that the remaining induced subgraph $G[V \setminus \Gamma]$ is disconnected or has at most one vertex. Let $\Gamma_G^{\min}$ denote any minimum vertex cut of G. For a disconnected graph, its minimum vertex cut is defined as the empty set. The *vertex connectivity* (VC) of G, denoted by $\kappa(G)$, is the size of a minimum vertex cut, i.e., $\kappa(G) = |\Gamma_G^{\min}|$.

Problem Definition 1. (**S**ize-Constrained Community Search Problem with **V**ertex **C**onnectivity, SVC). Given a graph $G = (V, E)$, a query vertex $q \in V$, and a size constraint $[l, h]$ where $l < h$ are positive integers, SVC aims to find an induced subgraph g of G that satisfies the following three conditions:

1) **Connectedness:** g is connected and contains q;
2) **Size Constraint:** $l \leq |V(g)| \leq h$;
3) **VC Optimality:** among all induced subgraphs satisfying the above two conditions, g has the largest vertex connectivity.

A connected induced subgraph g of G is a *candidate community* if it contains the query vertex q and satisfies the size constraint $l \leq |V(g)| \leq h$. An *optimal community* is a candidate community that attains the maximum VC among all candidate communities. We refer to this maximum value as the *optimal VC*. During the search, let k^* denote the largest VC among the candidate communities found so far. A *better community* is a candidate community whose VC exceeds k^*.

Example. Consider the SVC query on the graph in Figure 1(a) with $q = \text{James}$ and $[l, h] = [5, 7]$. The communities in panels (b) and (c) are both candidate communities. Suppose the community in panel (b) is the best found so far, so $k^* = 1$. The community in panel (c) has VC 2 and is therefore a better community. It is also optimal for this query.

We highlight three properties of SVC. First, candidate communities are non-hereditary. An induced subgraph of a candidate community may no longer contain the query vertex, remain connected, or satisfy the size lower bound. Second, the optimal community need not be unique. Consider the query with $q = \text{James}$ and $[l, h] = [2, 3]$ on the graph in Figure 1(a). Both $G[\{\text{James}, \text{Gerry}, \text{Peter}\}]$ and $G[\{\text{James}, \text{Lily}, \text{Peter}\}]$ are optimal communities. Each is a 3-clique with VC 2, the maximum possible for a graph with at most three vertices. Third, SVC is NP-hard, as established in Theorem 1.

Theorem 1. *The SVC problem is NP-hard.*

The proof of Theorem 1 is provided in our technical report.

III. Exact Algorithms for SVC

We develop two exact algorithms for SVC within a common branch-and-bound framework. We first introduce the shared framework and then present a baseline algorithm to establish the basic search procedure. Building on this baseline, we develop an advanced algorithm that exploits properties of vertex connectivity to identify and prune unpromising branches earlier.

A. Common Branch-and-Bound Framework

The framework maintains the best VC value found so far, denoted by k^*. A heuristic first attempts to find a candidate community to initialize k^*. The exact search then explores vertex selections, updating k^* whenever a better community is found and pruning branches that cannot improve it.

Branches and Search Space. The framework organizes the search into branches, each representing a set of possible candidate communities. A branch is described by a pair $B = (S, C)$ of disjoint vertex sets. The *selected set* S contains vertices that have already been chosen and must appear in every community considered in this branch. The *candidate set* C contains vertices that may still be added to S. Vertices outside $S \cup C$ have been excluded from the branch. Formally, a candidate community g belongs to B if and only if $S \subseteq V(g) \subseteq S \cup C$. Searching this branch therefore involves choosing vertices from C to extend S into a candidate community. Initially, the query vertex q must be included, while every other vertex remains available for selection. The initial branch is therefore $B_0 = (\{q\}, V \setminus \{q\})$. Every candidate community belongs to B_0, so this branch covers the entire search space of the query.

Algorithm 1: Heuristic initialization: SVC−Heu

Input: A graph $G = (V, E)$, a query vertex $q \in V$, and a size constraint $[l, h]$
Output: The best candidate community found, or $\emptyset$

```
S ← {q}, C ← V \ {q}, g ← ∅, k_heu ← −1;
if l = 1 then g ← G[S]; k_heu ← 0;
while |S| < h and C ≠ ∅ do
    v ← arg max_{u∈C} d(u, S);
    if d(v, S) = 0 then
        break;
    S ← S ∪ {v}, C ← C \ {v};
    if |S| ≥ l then
        k_S ← κ(G[S]);
        if k_S > k_heu then
            g ← G[S], k_heu ← k_S;
return g;
```

Heuristic Initialization. We use a greedy heuristic, SVC−Heu (Algorithm 1), to construct an initial candidate community by expanding a connected vertex set from q. At each step, the heuristic selects a vertex $v \in C$ with the largest number of neighbors in S. The vertex is added only if $d(v, S) > 0$, ensuring that $G[S]$ remains connected. After each addition, if $|S| \geq l$, the current subgraph is a candidate community. The heuristic then computes its VC and keeps the candidate community with the largest value found during the expansion. It stops when $|S| = h$ or no vertex in C has a neighbor in S.

When $l = 1$, the heuristic records $G[\{q\}]$ with VC 0. Otherwise, it can expand to size l whenever the query's connected component contains at least l vertices, because an adjacent vertex remains available until the component is exhausted. An empty result therefore certifies infeasibility, and the algorithm terminates. Otherwise, we initialize $k^* = \kappa(g^*) \geq 0$ and return immediately if $k^* = h - 1$, the largest possible VC.

The following property allows us to use the initial bound k^* to remove vertices that cannot belong to any better community.

Lemma 1. *For any graph g, its vertex connectivity is at most the degree of every vertex $v \in V(g)$, i.e., $\kappa(g) \leq d(v, V(g))$.*

Proof. Removing all neighbors of v either isolates v from the other remaining vertices or leaves only v. Thus, the neighbors of v form a vertex cut of size $d(v, V(g))$, giving $\kappa(g) \leq d(v, V(g))$. □

Lemma 1 allows us to remove vertices that cannot belong to any better community. If a vertex v has degree at most k^* in the current graph, its degree in any induced subgraph containing it is also at most k^*. Such a subgraph therefore cannot have VC greater than k^*, so v can be safely removed. Since removing

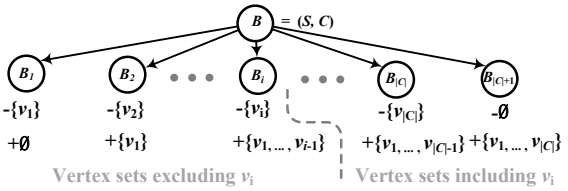


Fig. 2. Illustration of Sym−SE branching. "+" includes a candidate vertex in the selected set, whereas "−" excludes it from the branch.

a vertex may reduce the degrees of its neighbors, we repeat this process until no remaining vertex has degree at most k^*, before starting the exact search.

Set-Enumeration Branching. The branching strategy determines how each branch is divided into smaller sub-branches for further exploration. To handle the non-hereditary nature of candidate communities in SVC, we adopt the Symmetric Set-Enumeration branching (Sym−SE) strategy, which is suitable for non-hereditary subgraph search problems [37].

Given a branch $B = (S, C)$, let $\langle v_1, v_2, \ldots, v_{|C|}\rangle$ be an ordering of the candidate vertices. For each $1 \le i \le |C|$, Sym−SE creates a sub-branch $B_i = (S_i, C_i)$ that selects all vertices preceding v_i, excludes v_i, and leaves the subsequent vertices available for selection. Formally,

$$S_i = S \cup \{v_1, \ldots, v_{i-1}\}, \qquad C_i = C \setminus \{v_1, \ldots, v_i\}. \quad (1)$$

The last sub-branch $B_{|C|+1}$ includes all vertices in C, with $S_{|C|+1} = S \cup C$ and $C_{|C|+1} = \emptyset$. Thus, the original branch is divided into $|C| + 1$ sub-branches, as illustrated in Figure 2.

These sub-branches cover the search space of B without overlap. Any vertex set between S and $S \cup C$ either excludes a first vertex v_i in the given ordering, placing it in B_i, or includes all vertices in C, placing it in $B_{|C|+1}$. Exploring these sub-branches therefore preserves completeness.

The selected sets are nested, with $S_i \subseteq S_j$ whenever $i < j$. Thus, vertices selected in one sub-branch also appear in every later sub-branch. If these vertices already violate a necessary condition for a better community, and adding vertices cannot resolve the violation, all sub-branches containing them can be pruned together. Placing vertices that trigger this condition earlier in the ordering allows more sub-branches to be skipped.

Section III-B presents a baseline algorithm using a degree-based ordering. Section III-C then develops a VC-aware ordering and pruning strategy to improve search efficiency.

B. SVC−Enum: A Baseline Exact Method

We first present SVC−Enum, a baseline exact algorithm that applies the framework in Section III-A with an intuitive degree-based vertex ordering. It serves as a reference for assessing the efficiency gains from the VC-aware ordering and pruning strategies developed in Section III-C.

Degree-based Ordering. Given a branch $B = (S, C)$, SVC−Enum orders the candidate vertices by the number of neighbors they have in S:

$$\langle v_1, v_2, \ldots, v_{|C|}\rangle, \qquad d(v_1, S) \ge \cdots \ge d(v_{|C|}, S). \quad (2)$$

Algorithm 2: The baseline exact algorithm: SVC−Enum

Input: A graph $G = (V, E)$, a query vertex $q \in V$, and a size constraint $[l, h]$
Output: An optimal community g^*, or $\emptyset$ if none exists

```
// g* and k* are shared across all recursive calls
g* ← SVC-Heu(G, q, l, h);
if g* = ∅ then return ∅;
k* ← κ(g*);
if k* = h − 1 then return g* ;
if k* ≥ 0 then
    Iteratively remove from G vertices whose current degrees are at most k*;
if q ∈ V(G) then EnumRec({q}, V(G) \ {q}) ;
return g*;

Procedure EnumRec(S, C);
if |S| > h or |S| + |C| < l or k* = h − 1 then
    return;
if |S| ≥ l and G[S] is connected then
    k_S ← κ(G[S]);
    if k_S > k* then g* ← G[S]; k* ← k_S ;
if |S| = h or C = ∅ or k* = h − 1 then return;
Sort vertices in C according to Equation (2);
Generate sub-branches B_1, ..., B_{|C|+1} using Sym-SE;
foreach sub-branch B_i = (S_i, C_i) do
    EnumRec(S_i, C_i);
```

This ordering uses $d(v, S)$ as a simple measure of how closely a candidate vertex is connected to the selected set. Under Sym−SE, each sub-branch selects the vertices preceding its excluded vertex, so these selected vertices have at least as many neighbors in S as the remaining candidates. Although this ordering captures local connections to S, it does not directly account for the vertex-cut structure that determines VC.

Algorithm. Algorithm 2 presents SVC−Enum. It first calls SVC−Heu (Algorithm 1) to obtain an initial candidate community g^*. If the heuristic returns $\emptyset$, no feasible community exists, and the algorithm terminates. Otherwise, it initializes $k^* = \kappa(g^*)$ and returns g^* immediately if $k^* = h - 1$, the maximum possible VC under the size upper bound. The algorithm then applies Lemma 1, repeatedly removing vertices whose current degrees are at most k^*, since they cannot belong to any better community. If this process removes q, no better community containing the query can exist, so g^* is returned. Otherwise, the algorithm invokes EnumRec on the initial branch $(\{q\}, V(G) \setminus \{q\})$ in the reduced graph to search for further improvements.

At each branch (S, C), EnumRec first checks whether $G[S]$ is a candidate community, namely, whether it is connected and its size lies in $[l, h]$. If so, it computes $\kappa(G[S])$ and updates both g^* and k^* whenever a better community is found. If the branch can still be expanded, the procedure orders C according to Equation (2), generates sub-branches using Sym−SE, and recursively explores them. Branches with $|S| + |C| < l$ need no further exploration, because even selecting all remaining candidates cannot satisfy the lower size bound.

Correctness. SVC−Enum returns an optimal community whenever one exists. First, by Lemma 1, preprocessing does not discard any community with VC greater than k^*. Second, Sym−SE partitions the search space of each branch without overlap: every candidate community belonging to the parent branch belongs to exactly one generated sub-branch. The size-based pruning rules are also safe. A branch with $|S| > h$ cannot satisfy the upper size bound, while a branch with $|S|+|C| < l$ cannot satisfy the lower size bound. Early termination when $k^* = h - 1$ is valid because no candidate community can have a larger VC. Finally, the algorithm updates both g^* and k^* whenever it finds a better community. Thus, when the search terminates, no candidate community has VC greater than k^*, and g^* is optimal. If no candidate community exists, the algorithm returns $\emptyset$.

Time Complexity and Limitations. Let $n = |V|$ and $m = |E|$. We analyze the running time by bounding the cost per recursive call and the total number of calls. At each branch, computing $\kappa(G[S])$ involves at most $O(|S|^2)$ local connectivity computations, each taking $O(\sqrt{|S|}\,|E(G[S])|)$ time using a flow-based method [38]. This yields a worst-case upper bound of $O(|S|^{2.5}|E(G[S])|)$, which is at most $O(n^{2.5}m)$. The remaining operations, including checking connectedness and ordering candidate vertices, also fit within this bound.

The search tree contains $O(\min\{2^n, n^h\})$ recursive calls. Enumerating vertex subsets gives the $O(2^n)$ bound. For the alternative bound, a branch is expanded only when $|S| < h$. Since $q \in S$, there are at most $h-2$ other selected vertices, giving $O(n^{h-2})$ possible selected sets. Branches sharing a selected set form a chain of at most n nodes, each generating at most $n+1$ children, yielding $O(n^h)$ calls. With an $O(n^{2.5}m)$ cost per call, the overall running time is $O(n^{2.5}m\min\{2^n, n^h\})$, or $O^*(\min\{2^n, n^h\})$ when polynomial factors are suppressed.

Although exact, SVC−Enum has limited pruning power. Its degree-based ordering captures only the connections between candidate vertices and the selected set; it provides no bound on the VC achievable within a branch. Consequently, the algorithm may explore many branches that cannot improve the current best solution. This motivates the vertex-cut-based ordering and VC-aware pruning conditions developed in the next subsection.

C. SVC−BU: A VC-aware Bottom-Up Method

To strengthen the pruning power of SVC−Enum, we develop a more efficient exact algorithm, SVC−BU, that exploits the structural properties of vertex connectivity. We first derive a necessary condition for a branch to contain a better community. We then design a vertex ordering that works with Sym−SE to expose violations of this condition early, allowing multiple sub-branches to be pruned together.

A Necessary Condition for Better Communities. We begin by examining the relationship between graph size and vertex connectivity. Given a graph g, a *cut-based vertex partition* divides $V(g)$ into a vertex cut $F(g)$ and the remaining vertices $H(g) = V(g) \setminus F(g)$. Different choices of $F(g)$ may yield different partitions. Let $\mathcal{H}_g$ denote the collection of all remaining vertex sets obtained in this way. We define

$$\phi(g) = \max\{|H(g)| \mid H(g) \in \mathcal{H}_g\}. \tag{3}$$

Thus, $\phi(g)$ is the largest number of vertices that can remain after removing a vertex cut. Maximizing this number is equivalent to choosing a minimum vertex cut, denoted by $\Gamma_g^{\min}$. Therefore,

$$\phi(g) = |V(g)| - |\Gamma_g^{\min}| = |V(g)| - \kappa(g). \tag{4}$$

For graphs of the same size, a smaller ϕ thus corresponds to a larger VC. More importantly for our search, ϕ cannot decrease when an induced subgraph is expanded, as established below.

Lemma 2 (Monotonicity of ϕ)**.** *For any induced subgraph g of a graph G, we have $\phi(g) \le \phi(G)$.*

Proof. Let $\Gamma_g^{\min}$ be a minimum vertex cut of g. Removing $\Gamma_g^{\min}$ together with all vertices outside g from G leaves exactly $g[V(g)\setminus\Gamma_g^{\min}]$, which is disconnected or has at most one vertex. Hence, $\Gamma_g^{\min} \cup (V(G) \setminus V(g))$ is a vertex cut of G, and

$$\kappa(G) \le \kappa(g) + |V(G)| - |V(g)|.$$

Rearranging gives

$$|V(g)| - \kappa(g) \le |V(G)| - \kappa(G).$$

By Equation (4), this is equivalent to $\phi(g) \le \phi(G)$. □

Consider a branch $B = (S, C)$ and let k^* be the current best VC value. Any better community in B must have VC at least $k^* + 1$ and contain at most $\min\{h, |S \cup C|\}$ vertices. Consequently, its ϕ value must be smaller than the branch-specific threshold

$$\tau_B(k^*) = \min\{h, |S \cup C|\} - k^*. \tag{5}$$

Since every community in B contains S, the monotonicity of ϕ provides a necessary condition for extending S into a better community.

Lemma 3 (The Necessary Condition)**.** *If a branch $B = (S, C)$ contains a better community, then $\phi(G[S]) < \tau_B(k^*)$.*

Proof. Let g be a better community in B. Since $|V(g)| \le \min\{h, |S \cup C|\}$ and $\kappa(g) \ge k^* + 1$, Equation (4) gives

$$\phi(g) = |V(g)| - \kappa(g) \le \min\{h, |S \cup C|\} - (k^* + 1) < \tau_B(k^*).$$

Moreover, $G[S]$ is an induced subgraph of g. By Lemma 2, $\phi(G[S]) \le \phi(g)$, and hence $\phi(G[S]) < \tau_B(k^*)$. □

By Lemma 3, a branch $B = (S, C)$ can be pruned when $\phi(G[S]) \ge \tau_B(k^*)$. To prune multiple sub-branches together, we use a common threshold t inherited from B, such that every better community in B must satisfy $\phi(g) < t$. Suppose a sub-branch has selected a set S_i with $\phi(G[S_i]) \ge t$. Every later sub-branch also selects all vertices in S_i. By monotonicity, adding vertices cannot reduce ϕ, so none of these sub-branches can contain a better community. We can therefore prune them together, including the final sub-branch that selects all

candidates. This motivates the VCL ordering, which places vertices that trigger this pruning condition early.

Identifying Better Communities by Size. Before branching, we use the current threshold to directly construct a better community of maximum feasible size. Let $g_B = G[S \cup C]$, $p = \phi(g_B)$, and $M = \min\{h, |S \cup C|\}$. After the size checks and threshold update, $M \geq \max\{l, |S|\}$ and $t \leq M - k^*$. The following lemma shows that $p < t$ guarantees successful construction.

Lemma 4 (Direct Identification)**.** *Let $k^* \geq 0$, $M \geq \max\{l, |S|\}$, and $p < t \leq M - k^*$. Any set U satisfying $S \subseteq U \subseteq S \cup C$ and $|U| = M$ induces a better community of maximum feasible size in the branch.*

Proof. Since p and t are integers, $p+1 \leq t \leq M-k^*$. Together with size feasibility, this gives

$$M \geq \max\{l, |S|, p + k^* + 1\}. \tag{6}$$

By monotonicity, $\phi(G[U]) \leq p$, so $\kappa(G[U]) \geq M - p \geq k^* + 1 \geq 1$. Thus, $G[U]$ is connected, contains q, satisfies the size constraints, and improves the incumbent. No community in the branch can have more than M vertices. □

We construct U by adding any $M - |S|$ vertices from C to S. After updating the incumbent, $k^* \geq M - p$, so tightening t to $\min\{t, M - k^*\}$ yields $t \leq p$. Direct identification therefore needs to be performed only once per call.

The VCL Ordering. We now introduce *Vertex Cut Last* (VCL), an ordering strategy designed to prune multiple Sym-SE sub-branches after only a small number of candidate vertices have been selected. Given a branch $B = (S, C)$ and the current threshold t, we seek a small subset $C_{\text{sub}} \subseteq C$ such that

$$\phi(G[S \cup C_{\text{sub}}]) \geq t.$$

We place these vertices first in the candidate ordering. Once a sub-branch has selected all of them, monotonicity ensures that every community in that sub-branch has a ϕ value at least t, violating the necessary condition for a better community. Since all subsequent sub-branches also select these vertices, they can be pruned together.

To construct this prefix, VCL places candidates outside a vertex cut before those in the cut. Let $g_B = G[S \cup C]$, and consider a cut-based vertex partition $(F(g_B), H(g_B))$. Removing $F(g_B)$ separates the vertices in $H(g_B)$ into connected components. Suppose we select candidates from $H(g_B)$ so that, together with the already selected vertices outside the cut, they occupy at least two of these components. Then removing $F(g_B) \cap S$ disconnects the selected subgraph. If at least t selected vertices remain outside this cut, the selected subgraph has a ϕ value of at least t, triggering pruning.

A minimum vertex cut provides the largest possible set of vertices outside the cut. We therefore consider the partition

$$F_1(g_B) = \Gamma^{\min}_{g_B}, \qquad H_1(g_B) = V(g_B) \setminus F_1(g_B). \tag{7}$$

This partition satisfies $|H_1(g_B)| = \phi(g_B)$. After handling the case $\phi(g_B) < t$, we have $|H_1(g_B)| \geq t$. We can therefore use this partition to construct a prefix by considering both the number of selected vertices outside the cut and the components they occupy. The following lemma states sufficient conditions for such a prefix to trigger pruning.

Lemma 5. *Given a branch $B = (S, C)$ and a cut-based vertex partition $(F(g_B), H(g_B))$ of $g_B = G[S \cup C]$, let $C_{\text{sub}} \subseteq H(g_B) \cap C$ and $g_b = G[S \cup C_{\text{sub}}]$. If $(S \setminus F(g_B)) \cup C_{\text{sub}}$ contains vertices from at least two connected components of $G[H(g_B)]$, then $F(g_B) \cap S$ is a vertex cut of g_b, and*

$$\phi(g_b) \geq |(S \setminus F(g_B)) \cup C_{\text{sub}}|.$$

In particular, if the right-hand side is at least t, then $\phi(g_b) \geq t$.

Proof. Removing $F(g_B) \cap S$ from g_b leaves exactly the vertices in $(S \setminus F(g_B)) \cup C_{\text{sub}}$. These vertices belong to at least two distinct connected components of $G[H(g_B)]$, so the remaining graph is disconnected. Thus, $F(g_B) \cap S$ is a vertex cut of g_b. By the definition of ϕ, we have

$$\phi(g_b) \geq |V(g_b) \setminus (F(g_B) \cap S)| = |(S \setminus F(g_B)) \cup C_{\text{sub}}|.$$

The claimed bound follows. □

Based on Lemma 5, we construct the VCL ordering by considering how vertices in $S \setminus F_1(g_B)$ are distributed among connected components of $G[H_1(g_B)]$. Let c denote the number of connected components in $G[H_1(g_B)]$ that contain vertices from $S \setminus F_1(g_B)$. There are three cases.

Scenario 1: $c > 1$**.** The selected vertices outside the cut already occupy at least two connected components. We therefore only need to ensure that their number reaches t. Let

$$a = \max\{0, t - |S \setminus F_1(g_B)|\}. \tag{8}$$

Choose any a vertices from $H_1(g_B) \cap C$ to form $C_{\text{sub}} = \{v_1, \ldots, v_a\}$ and place them first:

$$\langle v_1, \ldots, v_a, v_{a+1}, \ldots, v_{|C|} \rangle. \tag{9}$$

The remaining candidates may appear in any order. By Lemma 5, the selected subgraph satisfies $\phi(G[S \cup C_{\text{sub}}]) \geq t$. If $a = 0$, the current branch can be pruned immediately.

Scenario 2: $c = 1$**.** The selected vertices outside the cut all lie in one connected component. To satisfy the component requirement, first choose a candidate $u \in H_1(g_B) \cap C$ from another component. Set

$$a = \max\{1, t - |S \setminus F_1(g_B)|\},$$

and choose another $a - 1$ vertices from $(H_1(g_B) \cap C) \setminus \{u\}$. This gives the prefix $C_{\text{sub}} = \{u, v_2, \ldots, v_a\}$ and the ordering

$$\langle u, v_2, \ldots, v_a, v_{a+1}, \ldots, v_{|C|} \rangle, \tag{10}$$

with the remaining candidates in any order. The selected vertices outside the cut now occupy at least two components and number at least t, so Lemma 5 applies.

Scenario 3: $c = 0$**.** All selected vertices belong to the cut, i.e., $S \subseteq F_1(g_B)$. We must therefore select candidates from at least two components of $G[H_1(g_B)]$. Choose vertices u and

v from different components, set $a = \max\{2, t\}$, and choose another $a-2$ vertices from $H_1(g_B)\backslash\{u, v\}$. Since $S \subseteq F_1(g_B)$, every vertex in $H_1(g_B)$ is a candidate. The resulting prefix is $C_{\text{sub}} = \{u, v, v_3, \ldots, v_a\}$, giving the ordering

$$\langle u, v, v_3, \ldots, v_a, v_{a+1}, \ldots, v_{|C|}\rangle, \tag{11}$$

where the remaining candidates may appear in any order. Again, the prefix satisfies both conditions in Lemma 5.

In all three scenarios, selecting the entire prefix guarantees $\phi(G[S \cup C_{\text{sub}}]) \geq t$. Under `Sym-SE`, sub-branch B_{a+1} and every subsequent sub-branch include this prefix. By monotonicity, none can contain a better community satisfying $\phi(g) < t$. Thus, `SVC-BU` only needs to generate and explore the first a sub-branches.

The Bottom-Up Algorithm. Algorithm 3 presents `SVC-BU`, which follows the same initialization and preprocessing steps as `SVC-Enum`. Its main difference lies in the recursive procedure `Branching-BU`, which combines the necessary condition with the `VCL` ordering to strengthen pruning.

At each branch $B = (S, C)$, `Branching-BU` first checks whether $G[S]$ is a better community and updates g^* and k^* accordingly. It then terminates branches that cannot be expanded within the size constraints, or stops searching if $k^* = h-1$. For each remaining branch, it updates the threshold to the smaller of the inherited value t and $\tau_B(k^*)$.

When $p = \phi(g_B) < t$, Lemma 4 guarantees that adding any $M - |S|$ candidate vertices to S produces a better community of maximum feasible size M. The procedure constructs this community, assigns it to g^*, and updates k^* to its actual VC. It then tightens the threshold to $t = \min\{t, \tau_B(k^*)\}$, allowing the improved lower bound to strengthen subsequent pruning. Since maximum size alone does not guarantee maximum VC, the procedure continues with the pruning test and explores the remaining sub-branches when necessary.

Once $\phi(g_B) \geq t$, the procedure checks whether $\phi(G[S]) \geq t$. If so, it prunes the branch. Otherwise, it constructs the `VCL` ordering and recursively explores only the first a `Sym-SE` sub-branches. All subsequent sub-branches include the entire prefix and can be pruned together as discussed earlier.

Correctness and Time Complexity. The following theorem establishes the exactness and time complexity of `SVC-BU`.

Theorem 2. *`SVC-BU` is an exact algorithm for `SVC` with time complexity*

$$O(n^{2.5} m \min\{\lambda_s^n, n^h\}),$$

where $s = h - (k_0 + 1)$, k_0 is the initial value of k^* after heuristic initialization, and $\lambda_s < 2$ is the largest positive real root of

$$x^{s+2} - 2x^{s+1} + 1 = 0.$$

For $s = 1, 2, 3$, the values of λ_s are approximately 1.618, 1.839, and 1.928, respectively.

Proof Sketch. The key to correctness is that a branch is discarded only when no choice of its remaining vertices can improve the current best community. In particular, once a selected prefix has a ϕ value at least t, adding more vertices cannot reduce that value. Every sub-branch containing this prefix can therefore be skipped safely. Since the remaining search covers all possibilities for improvement, the final community is optimal whenever a candidate community exists.

The same observation explains the efficiency gain. A short prefix that triggers pruning allows the algorithm to skip many sub-branches at once, and this saving is repeated throughout the search. Bounding the resulting search tree gives $O(\lambda_s^n)$ recursive calls, with $\lambda_s < 2$ determined by the branching recurrence. Together with the alternative bound $O(n^h)$ imposed by the size constraint and the $O(n^{2.5}m)$ cost per call, this yields the stated time complexity.

The complete correctness proof and complexity analysis are provided in our technical report. □

Algorithm 3: The bottom-up exact algorithm: `SVC-BU`

Input: A graph $G = (V, E)$, a query vertex $q \in V$, and a size constraint $[l, h]$
Output: An optimal community g^*, or $\emptyset$ if none exists

```
// g* and k* are shared across all recursive calls
g* ← SVC-Heu(G, q, l, h);
if g* = ∅ then return ∅;
k* ← κ(g*);
if k* = h − 1 then return g*;
Iteratively remove vertices of degree ≤ k* from G;
if q ∈ V(G) then
    t ← min{h, |V(G)|} − k*;
    Branching-BU({q}, V(G) \ {q}, t);
return g*;

Procedure: Branching-BU(S, C, t)
if |S| ∈ [l, h] and G[S] is connected then
    k_S ← κ(G[S]);
    if k_S > k* then g* ← G[S]; k* ← k_S;
if k* = h − 1 or |S| ≥ h then return;
if C = ∅ or |S| + |C| < l then return;
g_B ← G[S ∪ C]; p ← φ(g_B); M ← min{h, |S ∪ C|};
t ← min{t, τ_B(k*)};
if p < t then
    Choose any U with S ⊆ U ⊆ S ∪ C and |U| = M;
    g* ← G[U]; k* ← κ(g*);
    if k* = h − 1 then return;
    t ← min{t, τ_B(k*)};
if φ(G[S]) ≥ t then return;
Compute (F_1(g_B), H_1(g_B)) using Equation (7);
Construct the VCL ordering and prefix length a;
Generate the first a Sym-SE sub-branches B_1, ..., B_a;
foreach B_i = (S_i, C_i) do Branching-BU(S_i, C_i, t);
```

IV. Practical Acceleration Techniques

We develop three techniques to accelerate `SVC-BU`.

A. Fast Construction of the `VCL` Ordering

Constructing the `VCL` ordering does not always require a minimum vertex cut. The construction in Section III-C uses a minimum cut to maximize the number of vertices in $H(g_B)$. However, the prefix construction only requires a valid cut-based

partition with at least t vertices in $H(g_B)$ and vertices in at least two connected components. Any partition meeting these requirements can be used to construct a prefix that triggers pruning. This observation allows us to avoid an expensive minimum-cut computation when the branch contains a vertex of sufficiently small degree. Removing the neighbors of such a vertex isolates it while leaving enough vertices outside the cut. The following lemma formalizes this construction.

Lemma 6. *Given a branch $B = (S, C)$, let $g_B = G[S \cup C]$ and $t \geq 2$. If a vertex $v \in V(g_B)$ satisfies*

$$d(v, V(g_B)) \leq |V(g_B)| - t,$$

then

$$F_2(g_B) = N(v, V(g_B)), H_2(g_B) = V(g_B) \setminus F_2(g_B) \quad (12)$$

form a cut-based vertex partition with $|H_2(g_B)| \geq t$. This partition can be used in the VCL *ordering construction.*

Proof. Removing $N(v, V(g_B))$ leaves v isolated. Moreover,

$$|H_2(g_B)| = |V(g_B)| - d(v, V(g_B)) \geq t \geq 2.$$

Thus, at least one other vertex remains, and $G[H_2(g_B)]$ is disconnected. Hence, $F_2(g_B)$ is a valid vertex cut, and its remaining set contains at least t vertices, as required by the VCL ordering construction. □

To avoid computing a minimum vertex cut unnecessarily, we first perform the size checks, update t, and prune the branch if $\phi(G[S]) \geq t$. For each remaining branch, we check whether a vertex v satisfies $d(v, V(g_B)) \leq |V(g_B)| - t$. Since $\kappa(g_B) \leq d(v, V(g_B))$, this condition implies $\phi(g_B) \geq t$, so direct identification is not needed. We can then use the neighbors of v as the cut $F_2(g_B)$ and construct the ordering through Scenarios 1–3, without computing the exact value of $\phi(g_B)$. Finding such a vertex and constructing the partition and ordering take $O(|V(g_B)| + |E(g_B)|)$ time. If no vertex satisfies the degree condition, we compute a minimum vertex cut to obtain $\phi(g_B)$ and continue as in Algorithm 3, reusing the cut if ordering construction is needed.

Example. Consider the social network in Figure 1(a), with $q = \text{James}$, $[l, h] = [2, 4]$, $S = \{\text{James}\}$, $C = \{\text{Gerry}, \text{Lily}, \text{Mike}, \text{Robin}, \text{Frank}\}$, and $k^* = 1$. The branch graph has six vertices, and the threshold is $t = \tau_B(k^*) = \min\{4, 6\} - 1 = 3$. Frank has degree 2 in g_B, satisfying $2 \leq |V(g_B)| - t = 3$. Removing his two neighbors therefore leaves four vertices, including the isolated vertex Frank. This gives a valid partition with $|H_2(g_B)| = 4 \geq t$, which can be used to construct the VCL ordering according to Scenarios 1–3 without computing a minimum vertex cut.

B. Candidate Reduction by Missing-Neighbor Budgets

Before branching, we identify candidate vertices that cannot belong to a better community containing S. A bound on ϕ limits the number of missing adjacencies at each vertex. For each candidate, we temporarily add it to S and use the remaining allowance for missing adjacencies to bound how many more vertices can be included. The resulting size bound helps determine whether the candidate can participate in a better community.

For a vertex $u \in X$, define $\mu(u, X) = |X| - d(u, X)$. This count includes u itself, since a vertex is not adjacent to itself. The following lemma relates this count to ϕ.

Lemma 7. *For any graph g and any vertex $u \in V(g)$, we have $\mu(u, V(g)) \leq \phi(g)$.*

Proof. By Lemma 1, $\kappa(g) \leq d(u, V(g))$. Hence, $\mu(u, V(g)) = |V(g)| - d(u, V(g)) \leq |V(g)| - \kappa(g) = \phi(g)$. □

Let $t' = t - 1$ be the upper bound on ϕ for the communities considered under the current strict threshold $\phi(g) < t$. By Lemma 7, each vertex in such a community has a missing-neighbor count of at most t'. For a selected set T, define

$$\text{loss}(T) = \sum_{u \in T} \mu(u, T), \qquad \text{rest}(T) = |T|t' - \text{loss}(T).$$

Here, $\text{loss}(T)$ counts the missing adjacencies already present within T, while $\text{rest}(T)$ bounds the additional missing adjacencies that vertices in T can collectively accommodate. To evaluate a candidate $v \in C$, temporarily include it by setting $T = S \cup \{v\}$. If $\text{rest}(T) < 0$, no community containing T can satisfy $\phi \leq t'$, so v can be removed immediately.

Otherwise, adding a remaining candidate $x \in C \setminus \{v\}$ consumes one unit of budget for each vertex in T that is not adjacent to x. We denote this cost by

$$b(x, T) = |T| - d(x, T).$$

We use bucket sorting to order the remaining candidates by increasing integer cost $b(x, T)$. Let p_v be the largest number of candidates whose total cost fits within $\text{rest}(T)$, obtained by taking the cheapest candidates first. Then $U_v = \min\{h, |T| + p_v\}$ bounds the size of any target community containing T. This bound is optimistic because it ignores individual vertex budgets and missing adjacencies among the added candidates.

The size bound U_v also limits how many neighbors a selected vertex can have in a target community. Let $L_v = \max_{u \in T} \mu(u, T)$, and consider a vertex $u \in T$ attaining this maximum. Its missing-neighbor count is already L_v within T and cannot decrease as more vertices are added. Since any target community containing T has at most U_v vertices, the degree of u in that community is at most $U_v - L_v$. By Lemma 1, its VC is therefore also at most $U_v - L_v$.

Reduction Rule (RR). For a candidate vertex $v \in C$, let $T = S \cup \{v\}$. Remove v if $\text{rest}(T) < 0$ or $U_v - L_v \leq k^*$.

Lemma 8. *Any candidate vertex removed by RR cannot belong to a better community in a branch satisfying $\phi(g) \leq t'$.*

Proof. Any target community g containing $T = S \cup \{v\}$ must respect the missing-neighbor budget, so $\text{rest}(T) \geq 0$ and $|V(g)| \leq U_v$. Moreover, adding vertices cannot decrease the missing-neighbor counts in T. By Lemma 7, $\phi(g) \geq L_v$, and hence $\kappa(g) = |V(g)| - \phi(g) \leq U_v - L_v$. Thus, a negative budget rules out any target community containing T, while

$U_v - L_v \leq k^*$ rules out any better one. Both removal conditions are safe. □

Example. Consider the social network in Figure 1(a), with $t' = 2$, $q = \text{Frank}$, $[l, h] = [3, 6]$, $S = \{\text{Frank}\}$, $C = \{\text{Gerry}, \text{Lily}, \text{Mike}, \text{Robin}, \text{James}\}$, and $k^* = 2$. To evaluate Gerry, let $T = \{\text{Frank}, \text{Gerry}\}$. These two vertices are not adjacent, so each has a missing-neighbor count of 2, including itself. Consequently, $\text{loss}(T) = 4$, $\text{rest}(T) = 2 \times 2 - 4 = 0$, and $L_{\text{Gerry}} = 2$. Only candidates adjacent to both selected vertices can be added without exceeding this budget. At most two such candidates can be included, giving $U_{\text{Gerry}} = 4$. Since $U_{\text{Gerry}} - L_{\text{Gerry}} = 2 \leq k^*$, Gerry cannot belong to a better community under the current ϕ bound and is removed.

C. SVC−TD: A Top-Down Exact Variant

We introduce SVC−TD to strengthen early pruning by exploring tighter thresholds first. A branch is pruned when $\phi(G[S]) \geq t$, so a smaller t makes this condition easier to satisfy. In SVC−BU, the threshold follows the size bound and current incumbent through $\tau_B(k^*)$. A small initial k^* can therefore leave many unpromising branches to explore.

SVC−TD instead starts with a small threshold and gradually increases it, searching for better communities satisfying $\phi(g) < \theta$ at outer threshold θ. For communities of the same size, a smaller ϕ indicates higher vertex connectivity, so tighter thresholds prioritize more robust communities of that size. Finding a better community early raises k^* and strengthens subsequent pruning. As θ increases, the search also considers communities excluded by earlier thresholds, preserving completeness.

Algorithm 4 follows the initialization and preprocessing of SVC−BU and incorporates fast VCL construction and RR. The outer loop starts with $\theta = 2$ and increases it after each iteration. Within each branch, the local threshold t is bounded by θ and tightened using $\tau_B(k^*)$. When $p = \phi(g_B) < t$, Lemma 4 provides a better community of maximum feasible size. The algorithm updates g^* and k^* and immediately terminates that branch; otherwise, it continues with pruning and branching. The outer loop terminates when $\theta + k^* > h$, because any further improvement would require $\phi(g) < \theta - 1$, a range already covered by earlier iterations.

Correctness. SVC−TD returns an optimal community if one exists; see our technical report for the correctness proof.

Example. Consider Figure 1(a), with query vertex $q = \text{James}$, size constraint $[l, h] = [2, 4]$, and initial lower bound $k^* = 1$. The first iteration uses $\theta = 2$ and searches for better communities satisfying $\phi(g) < 2$. Since $\phi(g) = |V(g)| - \kappa(g) \geq 1$, these communities are cliques. In particular, $\{\text{James}, \text{Lily}, \text{Peter}, \text{Gerry}\}$ induces a four-vertex clique, with $\phi(g) = 1$ and $\kappa(g) = 3$. Once this community is found, $k^* = 3 = h - 1$, and the algorithm terminates because no community of at most four vertices can have higher vertex connectivity. This example illustrates how a small initial threshold can identify an optimal community before larger thresholds are explored.

Algorithm 4: The top-down exact algorithm: SVC−TD

Input: A graph $G = (V, E)$, a query vertex $q \in V$, and a size constraint $[l, h]$
Output: An optimal community g^*, or $\emptyset$ if none exists

```
Initialize and preprocess as in Algorithm 3, returning if no candidate community exists;
if k* = h − 1 or q ∉ V(G) then return g*;
θ ← 2;
while θ + k* ≤ h do
    Branching-TD({q}, V(G) \ {q}, θ);
    if k* = h − 1 then break;
    θ ← θ + 1;
return g*;
```

Procedure: Branching-TD(S, C, t)
Use the same steps as Branching-BU, incorporating RR and fast VCL construction as described in Section IV-A. The main difference is what happens after direct identification. When $p < t$, construct a community of size M containing S and update g^* and k^*, then return immediately instead of continuing the search within this branch. Otherwise, follow the same pruning and branching steps, using Branching-TD for all recursive calls.

Time Complexity Analysis. If initialization finds no feasible community or reaches the maximum VC $h - 1$, the algorithm returns immediately. Otherwise, let $k_0 < h - 1$ be the initial connectivity lower bound. The outer threshold starts at 2 and never exceeds $h - k_0$, giving at most $h - k_0 - 1$ iterations. Each iteration follows the branching framework of SVC−BU. Fast VCL construction and bucket-based RR require $O(n + m)$ and $O(|C|^2)$ time per branch, respectively, preserving the $O(n^{2.5}m)$ per-call bound. By Theorem 2, the total running time in this case is

$$O\big((h - k_0 - 1)n^{2.5}m \min\{\lambda_s^n, n^h\}\big),$$

where $s = h - (k_0 + 1)$.

V. Experiments

We evaluate the efficiency and scalability of our exact algorithms, the contribution of individual acceleration techniques, and the effectiveness of the SVC objective.

Experimental setup. Our algorithms are implemented in C++ and compiled with −O3 optimization. Runtime experiments are conducted on a machine with a 2.60GHz Intel CPU and 256GB memory, running Ubuntu 20.04.6, with a time limit of 7,200 seconds per query. Unless otherwise specified, we use size intervals $[l, h] \in \{[3, 6], [6, 9], [9, 12], [12, 15], [15, 18]\}$, following prior studies [3], [4]. For each size interval, we randomly sample 100 query vertices and use the same query set for all methods. We report mean running times, including preprocessing and heuristic initialization, with each timed-out query counted as 7,200 seconds. **Our code is available in [39].**

Datasets. We evaluate efficiency and scalability on 12 real-world graphs, summarized in Table I[1]. Following the practice

[1]UK2002 and Webbase are obtained from WebGraph [40]; the remaining graphs are from SNAP [41].

TABLE I
STATISTICS OF 12 REAL-WORLD DATASETS.

ID	Graph	n	m	ID	Graph	n	m
G1	Email	36K	183K	G7	BerkStan	685K	7M
G2	HepPh	34K	420K	G8	Gplus	108K	12M
G3	Epinions	131K	841K	G9	Flickr	2M	16M
G4	DBLP	317K	1M	G10	Orkut	3M	117M
G5	YouTube	1M	3M	G11	UK2002	18M	262M
G6	Google	876K	4M	G12	Webbase	118M	1B

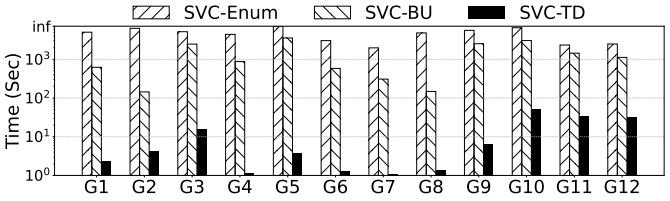
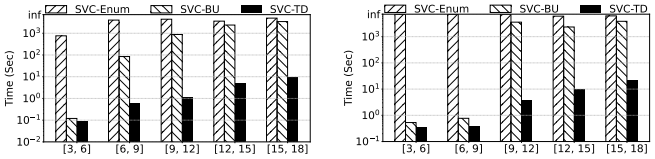


Fig. 3. Running time on all graphs with $[l, h] = [9, 12]$.



(a) DBLP (b) YouTube

Fig. 4. Running time across size intervals on DBLP and YouTube.

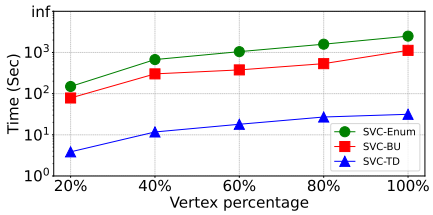


Fig. 5. Running time with increasing graph size on Webbase (G12), with $[l, h] = [9, 12]$.

of presenting detailed results on two representative datasets [3]–[5], we select DBLP (G4) and YouTube (G5) for comparisons across size settings. For effectiveness evaluation, we additionally use the Amazon co-purchasing graph with category-based ground-truth communities, described in Section V-C.

A. *Efficiency Evaluation*

We compare the running time of three exact algorithms:

- `SVC-Enum`: the baseline enumeration method (Algorithm 2).
- `SVC-BU`: the bottom-up search method (Algorithm 3).
- `SVC-TD`: the top-down search method (Algorithm 4).

Unless otherwise specified, both `SVC-BU` and `SVC-TD` use fast `VCL` construction and RR from Section IV, allowing us to examine the effect of their different threshold search strategies. All three algorithms use `SVC-Heu` (Algorithm 1) to initialize the best community and its connectivity lower bound.

Exp1: Running time on all datasets. Figure 3 compares the three algorithms on all 12 datasets with $[l, h] = [9, 12]$. This experiment examines whether the proposed methods improve efficiency across graphs with different sizes and structures. `SVC-TD` consistently achieves the lowest running time, with speedups of up to three orders of magnitude over `SVC-Enum`. On G8, for example, it reduces the running time from 4,824.20s to 1.39s. It also substantially outperforms `SVC-BU`, reducing the running time on G9 from 2,534.53s to 6.58s. Since both methods use fast `VCL` construction and RR, this comparison supports the benefit of exploring tighter thresholds first. Finding better communities early raises the connectivity lower bound and strengthens subsequent pruning. The consistent advantage across all datasets indicates that the efficiency improvement is not confined to a particular graph.

Exp2: Running time across size intervals. We next examine whether the efficiency advantage persists across different requested community sizes on DBLP and YouTube. Figure 4 shows that `SVC-TD` remains the fastest method in every tested setting. Running time generally increases as the intervals shift toward larger community sizes, which permit deeper recursive exploration. On DBLP, for example, `SVC-TD` takes 1.14s, 4.88s, and 9.05s for $[9, 12]$, $[12, 15]$, and $[15, 18]$, respectively. Although larger communities require more search effort, `SVC-TD` retains its efficiency advantage on both graphs. These results complement Exp1 by showing that the advantage holds across size settings as well as datasets.

Exp3: Scalability with graph size. We evaluate scalability on Webbase (G12), the largest dataset in Table I, using induced subgraphs containing increasing fractions of its vertices. We keep $[l, h] = [9, 12]$ throughout this experiment to examine graph growth under a fixed community-size requirement. Figure 5 shows that running time increases with graph size for all three algorithms. For example, `SVC-TD` takes 3.86s, 11.73s, and 18.04s on subgraphs containing 20%, 40%, and 60% of the vertices, respectively. It remains faster than `SVC-BU` and `SVC-Enum` at every tested fraction. Thus, even when the requested community size is fixed, searching a larger graph requires additional effort, but the proposed method maintains its relative efficiency advantage.

B. *Ablation and Sensitivity Studies*

We first assess the contributions of fast `VCL` construction and RR, and then examine runtime sensitivity to the upper and lower size bounds. The preceding comparison between `SVC-TD` and `SVC-BU`, both using fast `VCL` construction and RR, already evaluates the benefit of the top-down search strategy. We therefore focus the ablation study on the other two techniques, comparing the full `SVC-TD` algorithm with the following variants:

- `SVC-TD\FVCL`: uses the minimum-cut-based construction instead of fast `VCL` construction (Section IV-A), retaining the `VCL` ordering.
- `SVC-TD\R`: disables RR (Section IV-B).

Exp4: Contribution of acceleration techniques. Figure 6 compares `SVC-TD` with its two variants on all datasets under $[l, h] = [9, 12]$. Removing either technique increases running

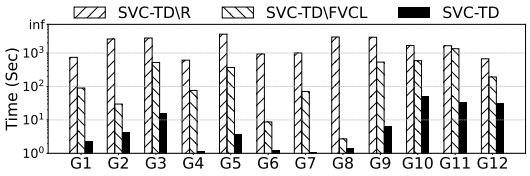
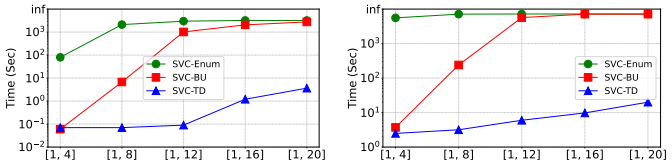


Fig. 6. Running time of SVC−TD and its ablation variants with $[l, h] = [9, 12]$.



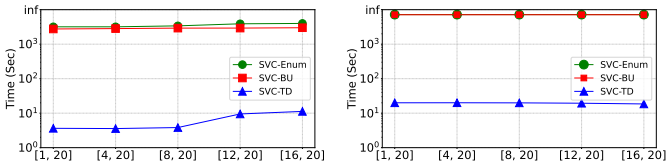
(a) DBLP (b) YouTube

Fig. 7. Running time with increasing size upper bound h, with $l = 1$.

time, confirming that both improve search efficiency. On UK2002 (G11), disabling fast VCL construction or RR raises the running time from 33.36s to 1,344.16s and 1,651.55s, respectively. Their relative impact varies across datasets. On Webbase (G12), the variant without RR takes 670.19s, compared with 191.80s for the variant without fast VCL construction, indicating a greater contribution from candidate reduction in this setting. These results support the complementary roles of the two techniques: fast VCL construction lowers ordering costs, while RR reduces the candidate set before branching.

Exp5: Running time with varying size upper bounds. We fix the lower bound at $l = 1$ and increase h to evaluate how allowing larger communities affects running time. Unlike Exp2, which shifts both bounds, this experiment enlarges the feasible size interval while retaining all previously eligible communities. Figure 7 shows that running time increases for all methods, most sharply for SVC−BU. On YouTube, its running time grows from 3.71s to 239.15s and 5,663.72s as h increases from 4 to 8 and 12, respectively. A larger upper bound allows more vertices to be selected, increasing the number of candidate combinations that may require exploration. The milder growth of SVC−TD suggests that its tighter initial thresholds and subsequent pruning help control the additional search cost.

Exp6: Running time with varying size lower bounds. We vary the lower bound l while keeping the upper bound h fixed to examine sensitivity to the minimum community size. Figure 8 shows that running time changes relatively little across the tested settings. With h fixed, changing l leaves the maximum number of selected vertices unchanged, although it affects community feasibility and size-based pruning. The results indicate that varying the lower bound has only a modest impact on running time across the tested settings. SVC−TD remains faster than both alternatives throughout, while SVC−BU often performs close to SVC−Enum. Together with Exp5, this experiment shows that running time is more sensitive to the upper size bound than to the lower bound in our evaluation.



(a) DBLP (b) YouTube

Fig. 8. Running time with varying size lower bound and a fixed upper bound.

C. Effectiveness Evaluation

We evaluate whether maximizing VC yields meaningful and robust communities under size constraints. Using Amazon product categories as ground truth, we examine both category recovery (Exp7) and its stability after vertex removals (Exp8). We compare SVC−TD with methods that optimize alternative community quality objectives, together with a learning-based community search method, as detailed below.

- SE−MN*: a branch-and-bound method that maximizes minimum degree under size constraints [26].
- SMA: the Sub-chain Merge Algorithm for maximizing local sketch modularity under size constraints [27].
- ST−Exa: an exact method for size-constrained community search with a support-based objective [5].
- ISCCP: a conductance-based method that seeks communities with low conductance under size constraints [4].
- CommunityAF: a learning-based method that searches for communities using examples, without explicit size constraints [42].

Exp7: Case study on recovering ground-truth communities. We conduct a case study on the Amazon co-purchasing graph[2] to evaluate whether size-constrained community search can recover semantically related product groups. Each vertex represents a product, and an edge connects two products that are frequently co-purchased. We choose this dataset because it provides both behavioral links for community search and category-based labels for evaluating the returned communities. These labels provide an external reference for comparing different SCS objectives beyond their respective structural scores. Specifically, the ground-truth communities are connected components within Amazon product categories, with groups of fewer than three products excluded.

We evaluate category recovery using F1-score under five size intervals from $[6, 9]$ to $[18, 21]$. Within each interval, all methods use the same query vertices sampled from the corresponding ground-truth communities. Following prior evaluations [10], [30], [31], for each returned community S, we compute its F1-score against every ground-truth community T as $2|S \cap T|/(|S| + |T|)$ and retain the maximum as its final score. Here, S and T denote vertex sets. We report the average of these scores over all queries for each size interval.

Table II(a) shows that SVC−TD achieves the highest F1-score across all five size intervals. For example, under $[15, 18]$, it

[2]https://snap.stanford.edu/data/com-Amazon.html. The graph contains approximately 335K vertices and 926K edges.

TABLE II
CATEGORY RECOVERY AND STABILITY ON AMAZON.

Size range	SVC−TD	SE−MN*	SMA	ST−Exa	ISCCP	CommunityAF
(a) Exp7: F1-score ↑						
$[6, 9]$	**0.54**	0.52	0.51	0.44	0.31	0.46
$[9, 12]$	**0.58**	0.56	0.54	0.46	0.35	0.47
$[12, 15]$	**0.63**	0.61	0.61	0.50	0.39	0.47
$[15, 18]$	**0.66**	0.64	0.59	0.52	0.42	0.48
$[18, 21]$	**0.61**	0.59	0.57	0.49	0.37	0.47
(b) Exp8: Mean absolute F1-score change ↓						
$[6, 9]$	**0.06**	0.17	0.19	0.07	0.12	0.11
$[9, 12]$	**0.03**	0.15	0.04	0.08	0.05	0.14
$[12, 15]$	**0.05**	0.12	0.09	0.14	0.08	0.10
$[15, 18]$	**0.06**	0.13	0.14	0.19	0.10	0.13
$[18, 21]$	0.08	0.18	0.15	0.04	0.07	**0.02**

obtains 0.66, compared with 0.64 for the strongest competitor, SE−MN*. These modest but consistent gains suggest that maximizing vertex connectivity can help recover category-aligned product groups under size constraints. One possible explanation is that optimizing VC favors groups supported by redundant co-purchase paths, limiting reliance on individual products to maintain connectivity. Such structural redundancy may help distinguish coherent product groups from groups joined through a few connecting products. The results support the usefulness of SVC for category recovery on this dataset.

Exp8: Stability under vertex removal. Since SVC maximizes vertex connectivity to improve robustness to vertex failures, we examine whether this objective also helps preserve category recovery after vertex removals. Using the communities returned in Exp7, we conduct ten trials per community. In each trial, we uniformly select and remove two distinct non-query vertices from the original community. We retain only the connected component containing the query vertex, so the evaluation captures both the direct loss of removed vertices and any additional loss caused by fragmentation. This setting tests whether the structural robustness promoted by SVC translates into more stable category recovery by limiting the loss of surviving vertices disconnected from the query. We recompute its F1-score using the best-match evaluation in Exp7, selecting the best-matching ground-truth category independently before and after removal. For each trial, we compute the absolute F1-score change relative to the original community, then average these changes over the ten trials and all queries used in Exp7. Smaller values indicate more stable category recovery.

Table II(b) shows that SVC−TD achieves the smallest mean absolute F1-score change in four of the five size intervals. Its average across the five intervals is 0.056, compared with 0.084–0.150 for the competing methods. CommunityAF performs best under $[18, 21]$, whereas SVC−TD has the smallest change overall. Combined with the higher F1-scores in Exp7, these results suggest that SVC−TD provides both accurate and stable category recovery under the tested perturbations.

Vertex connectivity helps limit the impact of vertex removals on the query community. When $\kappa(g) \geq 3$, removing any two non-query vertices leaves the remaining community connected. Thus, all surviving products remain reachable from the query and are retained for F1-score evaluation. By favoring communities with higher vertex connectivity, SVC−TD reduces their vulnerability to fragmentation, helping preserve category recovery after vertex removal.

VI. RELATED WORK

Community search (CS) includes size-unconstrained and size-constrained settings. Size-unconstrained studies consider simple graphs [7]–[10], [13], [37], [43]–[48], as well as directed [49], dynamic [50], temporal [51], attributed [52]–[54], signed [55], and heterogeneous graphs [56]. These studies address diverse graph structures and quality objectives without enforcing a user-specified size interval. Comprehensive surveys are available in [57], [58].

Size-constrained CS imposes a single size bound, a fixed size, or simultaneous lower and upper bounds [6], [14]–[21]. Our work focuses on the latter setting, which controls both the minimum and maximum community size. Under both bounds, SCS maximizes minimum degree [3], while SE−MN* improves search efficiency through branch-and-bound techniques [26]. Other studies optimize triangle-support-based measures [5], conductance [4], or local sketch modularity [27]. These objectives emphasize cohesion, separability, or modularity, without directly maximizing tolerance to vertex failures.

Connectivity-based studies address community robustness. Edge connectivity [59], [60] and vertex connectivity (VC) [10], [30], [32]–[34], [38] measure resistance to edge and vertex removals, respectively. VC-based studies include enumerating highly connected components and searching for subgraphs under prescribed connectivity requirements. Such requirements treat connectivity as a feasibility condition, whereas our formulation maximizes it within a size budget.

A closely related problem is graph downsizing [36], which maximizes VC in a fixed-size induced subgraph without requiring a designated query vertex. Its nesting-tree framework uses a heuristic downsizing subroutine, with optimality guaranteed when the subproblem is solved exactly. Our SVC formulation requires query inclusion and a user-specified size interval, so a highly connected subgraph elsewhere in the graph is not a valid answer. We develop exact branch-and-bound algorithms that combine size-dependent pruning with vertex-cut-guided ordering to eliminate multiple unpromising branches together.

VII. CONCLUSION

In this paper, we studied SVC, which maximizes vertex connectivity under community-size bounds to optimize robustness to vertex failures. We developed exact branch-and-bound algorithms with heuristic initialization, vertex-cut-based ordering, pruning, and top-down search. Experiments on 12 real-world graphs show speedups of up to three orders of magnitude over the baseline. On Amazon, SVC−TD achieves the highest category-recovery F1-score across all tested intervals and the smallest mean absolute F1-score change in four of five intervals after two non-query vertex removals. These results support the efficiency of our algorithms and the effectiveness of the SVC objective. Future work includes parallelizing SVC−TD.

REFERENCES

[1] G. Palla, I. Derényi, I. Farkas, and T. Vicsek, "Uncovering the overlapping community structure of complex networks in nature and society," *Nature*, vol. 435, no. 7043, pp. 814–818, 2005.

[2] M. Sozio and A. Gionis, "The community-search problem and how to plan a successful cocktail party," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining (SIGKDD)*, 2010, pp. 939–948.

[3] K. Yao and L. Chang, "Efficient size-bounded community search over large networks," *Proceedings of the VLDB Endowment*, vol. 14, no. 8, pp. 1441–1453, 2021.

[4] Y. He, L. Lin, P. Yuan, R. Li, T. Jia, and Z. Wang, "Ccss: Towards conductance-based community search with size constraints," *Expert Systems with Applications*, vol. 250, p. 123915, 2024.

[5] F. Zhang, H. Guo, D. Ouyang, S. Yang, X. Lin, and Z. Tian, "Size-constrained community search on large networks: An effective and efficient solution," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 1, pp. 356–371, 2023.

[6] C. Zong, P. Gong, X. Zhang, T. Qiu, A. Zhang, and M.-x. Wang, "Efficient size-constrained (k, d)-truss community search," in *Proceedings of the International Conference on Advanced Data Mining and Applications (DASFAA)*, 2023, pp. 405–420.

[7] X. Xie, S. Liu, J. Zhang, S. Han, W. Wang, and W. Yang, "Efficient community search based on relaxed k-truss index," in *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2024, pp. 1691–1700.

[8] Q. Dai, R. Li, D. Cui, and G. Wang, "Theoretically and practically efficient maximum defective clique search," *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*, vol. 2, no. 4, pp. 1–27, 2024.

[9] X. Xu, H. Liu, X. Lv, Y. Wang, and D. Li, "An efficient and exact algorithm for locally h-clique densest subgraph discovery," *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*, vol. 2, no. 6, pp. 1–26, 2024.

[10] Y. Liu, H. Huang, K. Yu, S. Liu, and C. Long, "Efficient maximum s-bundle search via local vertex connectivity," *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*, vol. 3, no. 1, pp. 1–27, 2025.

[11] J. She, Y. Tong, L. Chen, and T. Song, "Feedback-aware social event-participant arrangement," in *Proceedings of the ACM on Management of Data (SIGMOD)*, 2017, pp. 851–865.

[12] P. Manchanda, G. Packard, and A. Pattabhiramaiah, "Social dollars: The economic impact of customer participation in a firm-sponsored online customer community," *Marketing Science*, vol. 34, no. 3, pp. 367–387, 2015.

[13] L. Chang, "Efficient maximum k-defective clique computation with improved time complexity," *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*, vol. 1, no. 3, pp. 1–26, 2023.

[14] B. Liu, F. Zhang, W. Zhang, X. Lin, and Y. Zhang, "Efficient community search with size constraint," in *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*, 2021, pp. 97–108.

[15] Y. Liu, B. Yan, B. Zhao, H. Su, Y. Chen, and M. Witbrock, "Efficient size-prescribed k-core search," in *Proceedings of the International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, 2023, pp. 271–275.

[16] K. Zhou, J. Xin, J. Chen, X. Zhang, B. Wang, and Z. Wang, "Effective and efficient community search with size constraint on bipartite graphs," *Information Sciences*, vol. 647, p. 119511, 2023.

[17] Y. Zhang, K. Wang, W. Zhang, W. Ni, and X. Lin, "Size-bounded community search over large bipartite graphs." in *Proceedings of the International Conference on Extending Database Technology (EDBT)*, 2024, pp. 320–331.

[18] W. Cui, Y. Xiao, H. Wang, and W. Wang, "Local search of communities in large graphs," in *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*, 2014, pp. 991–1002.

[19] C. Li, F. Zhang, Y. Zhang, L. Qin, W. Zhang, and X. Lin, "Efficient progressive minimum k-core search," *Proceedings of the VLDB Endowment*, vol. 13, no. 3, pp. 362–375, 2019.

[20] Q. Zhang and S. Liu, "Efficient exact minimum k-core search in real-world graphs," in *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*, 2023, pp. 3391–3401.

[21] M. Li, R. Borovica-Gajic, F. M. Choudhury, N. Cui, and L. Ding, "Maximal size constraint community search over bipartite graphs," *Knowledge-Based Systems*, vol. 297, p. 111961, 2024.

[22] T. Lappas, K. Liu, and E. Terzi, "Finding a team of experts in social networks," in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2009, pp. 467–476.

[23] J. S. Mueller, "Why individuals in larger teams perform worse," *Organizational Behavior and Human Decision Processes*, vol. 117, no. 1, pp. 111–124, 2012.

[24] V. Spirin and L. A. Mirny, "Protein complexes and functional modules in molecular networks," *Proceedings of the National Academy of Sciences*, vol. 100, no. 21, pp. 12 123–12 128, 2003.

[25] G. D. Bader and C. W. Hogue, "An automated method for finding molecular complexes in large protein interaction networks," *BMC Bioinformatics*, vol. 4, no. 1, p. 2, 2003.

[26] Y. Liu, H. Huang, K. Yu, S. Liu, C. Long, and Z. Gu, "Efficient size-bounded community search, revisited: Frameworks for practical improvements," *Proceedings of the ACM on Management of Data (SIGMOD)*, vol. 3, no. 6, pp. 1–26, 2025.

[27] D. Kim, T. Han, K. Feng, J. Kim, and S. Yoon, "LMSC: Local sketch modularity optimisation for size-constrained community search in networks," *Proceedings of the ACM on Management of Data (SIGMOD)*, vol. 4, no. 1, pp. 1–26, 2026.

[28] S. Freitas, D. Yang, S. Kumar, H. Tong, and D. H. Chau, "Graph vulnerability and robustness: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 6, pp. 5915–5934, 2022.

[29] W. Abbas, A. Laszka, and X. Koutsoukos, "Improving network connectivity and robustness using trusted nodes with application to resilient consensus," *IEEE Transactions on Control of Network Systems*, vol. 5, no. 4, pp. 2036–2048, 2017.

[30] Y. Li, Y. Zhao, G. Wang, F. Zhu, Y. Wu, and S. Shi, "Effective k-vertex connected component detection in large-scale networks," in *Proceedings of the International Conference on Database Systems for Advanced Applications (DASFAA)*, 2017, pp. 404–421.

[31] Y. Li, G. Wang, Y. Zhao, F. Zhu, and Y. Wu, "Towards k-vertex connected component discovery from large networks," *World Wide Web*, vol. 23, pp. 799–830, 2020.

[32] S. Hu, Y. Zhou, M. Xiao, Z.-H. Fu, and Z. Lü, "Listing maximal k-relaxed-vertex connected components from large graphs," *Information Sciences*, vol. 620, pp. 67–83, 2023.

[33] Y. Zhou, W. Lin, J.-K. Hao, M. Xiao, and Y. Jin, "An effective branch-and-bound algorithm for the maximum s-bundle problem," *European Journal of Operational Research*, vol. 297, no. 1, pp. 27–39, 2022.

[34] H. Liu, Y. Wang, X. Xu, and D. Li, "Bottom-up k-vertex connected component enumeration by multiple expansion," in *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*, 2024, pp. 3000–3012.

[35] Y. Liu, H. Huang, K. Yu, S. Liu, and C. Long, "Minimum k-vertex connected graph search," *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 7, pp. 4159–4165, 2025.

[36] Y. Liu, M. Zhang, J. Liu, and Y. Zhou, "Graph downsizing: a fast heuristic for querying closely connected subgraphs," *Journal of Heuristics*, vol. 32, 2026, article 9.

[37] K. Yu and C. Long, "Fast maximal quasi-clique enumeration: A pruning and branching co-design approach," *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*, vol. 1, no. 3, pp. 1–26, 2023.

[38] D. Wen, L. Qin, Y. Zhang, L. Chang, and L. Chen, "Enumerating k-vertex connected components in large graphs," in *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*, 2019, pp. 52–63.

[39] "SVC-TD: Source code," https://github.com/liubufan1998/Size-Constrained-VC-Community-Search, 2026.

[40] P. Boldi and S. Vigna, "The webgraph framework i: Compression techniques," in *Proceedings of the International Conference on World Wide Web (WWW)*, 2004, pp. 595–602.

[41] J. Leskovec and A. Krevl, "Snap datasets: Stanford large network dataset collection," 2014.

[42] J. Chen, Y. Xia, and J. Gao, "CommunityAF: An example-based community search method via autoregressive flow," *Proceedings of the VLDB Endowment*, vol. 16, no. 10, pp. 2565–2577, 2023.

[43] L. Chang and K. Yao, "Maximum k-plex computation: Theory and practice," *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*, vol. 2, no. 1, pp. 1–26, 2024.

[44] K. Yu and C. Long, "Maximum k-biplex search on bipartite graphs: A symmetric-BK branching approach," *Proceedings of the ACM on Management of Data (SIGMOD)*, vol. 1, no. 1, pp. 1–26, 2023.

[45] S. Gao, K. Yu, S. Liu, and C. Long, "Maximum k-plex search: An alternated reduction-and-bound method," *Proceedings of the VLDB Endowment*, vol. 18, no. 2, pp. 363–376, 2024.

[46] L. Chang, "Maximum defective clique computation: Improved time complexities and practical performance," *Proceedings of the VLDB Endowment*, vol. 18, no. 2, pp. 200–212, 2024.

[47] H. Xia, K. Yu, S. Liu, C. Long, and X. Zhou, "Maximum degree-based quasi-clique search via an iterative framework," in *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (SIGKDD)*, 2025, pp. 3285–3296.

[48] Y. Zhou, Q. Guo, Y. Fang, and C. Ma, "A counting-based approach for efficient k-clique densest subgraph discovery," *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*, vol. 2, no. 3, p. 119, 2024.

[49] Q. Liu, M. Zhao, X. Huang, J. Xu, and Y. Gao, "Truss-based community search over large directed graphs," in *Proceedings of the ACM on Management of Data (SIGMOD)*, 2020, pp. 2183–2197.

[50] Z. Lu, Y. Zhu, M. Zhong, and J. X. Yu, "On time-optimal (k, p)-core community search in dynamic graphs," in *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*, 2022, pp. 1396–1407.

[51] R.-H. Li, J. Su, L. Qin, J. X. Yu, and Q. Dai, "Persistent community search in temporal networks," in *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*, 2018, pp. 797–808.

[52] X. Huang and L. V. Lakshmanan, "Attribute truss community search," *Proceedings of the VLDB Endowment*, p. 949–960, 2017.

[53] Y. Zhou, Y. Fang, W. Luo, and Y. Ye, "Influential community search over large heterogeneous information networks," *Proceedings of the VLDB Endowment*, vol. 16, no. 8, pp. 2047–2060, 2023.

[54] Z. Chen, W. Shao, and W. Liu, "Range constrained group query on attribute social graph," *Distributed and Parallel Databases*, vol. 42, no. 3, pp. 337–375, 2024.

[55] R. Sun, C. Chen, X. Wang, Y. Zhang, and X. Wang, "Stable community detection in signed social networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 10, pp. 5051–5055, 2020.

[56] Y. Fang, Y. Yang, W. Zhang, X. Lin, and X. Cao, "Effective and efficient community search over large heterogeneous information networks," *Proceedings of the VLDB Endowment*, vol. 13, no. 6, pp. 854–867, 2020.

[57] Y. Fang, X. Huang, L. Qin, Y. Zhang, W. Zhang, R. Cheng, and X. Lin, "A survey of community search over big graphs," *The VLDB Journal*, vol. 29, pp. 353–392, 2020.

[58] X. Huang, L. V. S. Lakshmanan, and J. Xu, *Community Search over Big Graphs*. Morgan & Claypool Publishers, 2019.

[59] L. Chang, J. X. Yu, L. Qin, X. Lin, C. Liu, and W. Liang, "Efficiently computing k-edge connected components via graph decomposition," in *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*, 2013, pp. 205–216.

[60] L. Chang, X. Lin, L. Qin, J. X. Yu, and W. Zhang, "Index-based optimal algorithms for computing steiner components with maximum connectivity," in *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*, 2015, pp. 459–474.

APPENDIX A
PROOFS AND DETAILED ANALYSIS

A. NP-Hardness of SVC

Proof of Theorem 1. We prove the theorem by considering the decision version of SVC. Given a graph $G = (V, E)$, a query vertex $q \in V$, a size constraint $[l, h]$, and an integer τ, the decision version asks whether there exists a connected induced subgraph $g = G[S]$ such that $q \in S$, $l \le |S| \le h$, and $\kappa(g) \ge \tau$. This problem belongs to NP, since a vertex set S serves as a certificate, and one can verify in polynomial time whether $q \in S$, whether $l \le |S| \le h$, whether $G[S]$ is connected, and whether $\kappa(G[S]) \ge \tau$ using polynomial-time algorithms for vertex connectivity.

We show NP-hardness by a polynomial-time reduction from the CLIQUE problem. Given an instance $(G = (V, E), r)$ of CLIQUE, which asks whether G contains a clique of size r, we construct a graph $G' = (V', E')$ by adding a new query vertex $q^\star \notin V$ and connecting it to every vertex in V. Formally, $V' = V \cup \{q^\star\}$ and $E' = E \cup \{(q^\star, v) \mid v \in V\}$. We then construct an SVC decision instance on G' with query vertex $q^\star$, size constraint $[l, h] = [r, r+1]$, and connectivity threshold $\tau = r$.

We now prove the correctness of the reduction. If G contains a clique C of size r, then $G'[C \cup \{q^\star\}]$ is a complete graph on $r + 1$ vertices. Hence, its vertex connectivity is r, and it is a feasible "yes" solution to the constructed SVC decision instance. Conversely, suppose the constructed SVC instance has a "yes" solution $G'[S]$. Then $q^\star \in S$ and $\kappa(G'[S]) \ge r$. Since $\kappa(G'[S]) \le |S| - 1$ and $|S| \le r+1$, it follows that $|S| = r+1$. For any graph with $r + 1$ vertices, its vertex connectivity is at most its minimum degree, and its minimum degree is at most r. Therefore, $\kappa(G'[S]) \ge r$ implies that every vertex in $G'[S]$ has degree exactly r, and thus $G'[S]$ must be a complete graph on $r + 1$ vertices. Since $q^\star$ is the only added vertex, the remaining r vertices in $S \setminus \{q^\star\}$ must be pairwise adjacent in the original graph G. Thus, $S \setminus \{q^\star\}$ forms a clique of size r in G.

The reduction is clearly polynomial, and the CLIQUE instance is a "yes" instance if and only if the constructed SVC decision instance is a "yes" instance. Therefore, the decision version of SVC is NP-complete, and the optimization version of SVC is NP-hard. □

B. Correctness and Complexity of SVC−BU

Proof of Theorem 2. **Correctness.** We first prove the correctness of SVC−BU. The algorithm follows the branch-and-bound framework and systematically explores the search space through Sym−SE branching. During the search, SVC−BU prunes a branch only when this branch can be proven not to contain any better community. Specifically, the preprocessing step is safe by Lemma 1; the direct identification step is safe by Lemma 4; and the branch pruning condition based on $\phi(G[S])$ is safe by Lemma 3. For batch pruning, all children are tested against the parent threshold t. Once a selected prefix has $\phi \ge t$, every subsequent selected set also has $\phi \ge t$ by monotonicity. This includes the final child selecting all candidates; no containment between sibling available-vertex sets is needed.

Therefore, every branch that may contain an optimal community is either explored or safely handled by a correctness-preserving rule.

We also note that several corner cases do not affect correctness. For example, if the selected set S cannot be connected within $G[S \cup C]$, then no candidate community in the current branch can contain all vertices in S, and the branch can be safely terminated. If $G[S \cup C]$ is disconnected but all vertices in S lie in the same connected component, then only the component containing S needs to be retained, since vertices in other components cannot help form a connected community containing S. These operations only remove vertices or branches that cannot contribute to any feasible improvement. Moreover, SVC−BU updates g^* only when an actual candidate community with a larger VC is found. Hence, when the recursion terminates, no unexplored branch contains a community with VC larger than $\kappa(g^*)$, and g^* is an optimal community.

Time complexity. We next analyze the time complexity. The running time of SVC−BU is determined by two factors: the time cost of each recursive call and the total number of recursive calls. For each recursive call, the dominant operation is computing a minimum vertex cut, which is used to obtain $\phi(g_B)$ and construct the VCL ordering. Using a flow-based algorithm for vertex connectivity [38], this cost is bounded by $O(n^{2.5}m)$.

It remains to bound the number of recursive calls. Let k_0 be the initial value of k^* after heuristic initialization, and define $s = h - (k_0 + 1)$ as in Theorem 2. Every better community has $\phi(g) \le s$. The pruning test uses the strict condition $\phi(g) < t$, so the threshold can be one larger than this bound. In particular, since $k^* \ge k_0$ throughout the search,

$$t \le \tau_B(k^*) = \min\{h, |S \cup C|\} - k^* \le h - k_0 = s + 1. \quad (13)$$

If $s = 0$, the initial community already attains the upper bound $h - 1$, and the algorithm returns without branching. We therefore consider $s \ge 1$. At a branch that reaches the ordering construction, $t \ge 2$, since $\phi(G[S]) \ge 1$ and a branch with $\phi(G[S]) \ge t$ is pruned.

Let $r = |S \setminus F_1(g_B)|$ and let a be the prefix length constructed by VCL. The three scenarios give the following bounds.

Scenario 1: Vertices in $S \setminus F_1(g_B)$ are distributed across more than one component of $G[H_1(g_B)]$. Here, $r \ge 2$, and the construction sets $a = \max\{0, t - r\}$. Consequently,

$$0 \le a \le s - 1. \quad (14)$$

When $a = 0$, the branch is pruned without generating any recursive calls.

Scenario 2: Vertices in $S \setminus F_1(g_B)$ are in a single component of $G[H_1(g_B)]$. Here, $r \ge 1$. At least one candidate from another component is required, so $a = \max\{1, t - r\}$. Thus,

$$1 \le a \le s. \quad (15)$$

Scenario 3: Vertices in $S \setminus F_1(g_B)$ are in no component of $G[H_1(g_B)]$. Here, $r = 0$, and the construction sets $a = \max\{2, t\} = t$. Therefore,

$$2 \le a \le s + 1. \tag{16}$$

Only the first a `Sym-SE` sub-branches are explored. Sub-branch B_i has $|C| - i$ candidates, and every sub-branch after B_a is pruned. There is therefore no additional recursive call beyond these a sub-branches. Let $T(z)$ bound the number of calls in a recursion subtree with at most z candidates, over all three scenarios. Including the current call, the common worst-case recurrence is

$$T(z) \le 1 + \sum_{i=1}^{\min\{s+1,z\}} T(z-i), \qquad T(0) = 1. \tag{17}$$

This bound remains valid when recursive calls use different scenarios, since each has prefix length at most $s+1$. For $s \ge 1$, the recurrence yields $T(z) = O(\lambda_s^z)$, where $\lambda_s > 1$ satisfies

$$\sum_{i=1}^{s+1} \lambda_s^{-i} = 1. \tag{18}$$

Equivalently, λ_s is the largest positive real root of

$$x^{s+2} - 2x^{s+1} + 1 = 0. \tag{19}$$

For $s = 1, 2, 3, 4, 5$, its values are approximately 1.618, 1.839, 1.928, 1.966, and 1.984, respectively. Since $|C| \le n$, the total number of recursive calls is $O(\lambda_s^n)$. In addition, since `SVC-BU` never needs to explore vertex sets with size larger than h, the number of recursive calls is also bounded by $O(n^h)$. Therefore, the total number of recursive calls is $O(\min\{\lambda_s^n, n^h\})$. Multiplying it by the per-recursion cost $O(n^{2.5}m)$ gives the overall time complexity

$$O(n^{2.5} m \min\{\lambda_s^n, n^h\}). \tag{20}$$

This completes the proof. □

C. Correctness of `SVC-TD`

Proof. Initialization either reports infeasibility or supplies an incumbent with $k^* \ge 0$, including the singleton community when $l = 1$. We prove by induction over the outer threshold θ that, before each iteration, no community with $\phi(g) < \theta - 1$ improves the incumbent. This holds at $\theta = 2$ because every nonempty graph has $\phi \ge 1$. Increasing k^* preserves this invariant.

Within an iteration, the local threshold satisfies $t \le \theta$. Tightening it to $\tau_B(k^*)$ preserves every better community in the branch by Lemma 3. Batch pruning uses the common parent threshold and nested selected sets, as in `SVC-BU`. Fast construction is also safe: a vertex satisfying $d(v, V(g_B)) \le |V(g_B)| - t$ certifies $p \ge t$, so direct identification cannot apply. Its neighborhood supplies a valid cut for the same prefix construction without evaluating p. If the degree check fails, the algorithm computes p and follows the original steps. It remains to justify termination when $p = \phi(g_B) < t$. Write $M = \min\{h, |S \cup C|\}$ and let k be the incumbent value before direct identification.

The size checks ensure $M \ge \max\{l, |S|\}$, and the threshold update gives $t \le M - k$. Therefore, $p < t$ implies $M \ge p + k + 1$, so Lemma 4 guarantees successful construction of $g' = G[U]$ with $|U| = M$ and $\kappa(g') \ge M - p > k$. Any community g in the branch that could still improve the incumbent has $\phi(g) \ge \theta - 1$ by the induction hypothesis. Since $p < t \le \theta$, we obtain

$$\kappa(g) \le M - (\theta - 1) \le M - p \le \kappa(g').$$

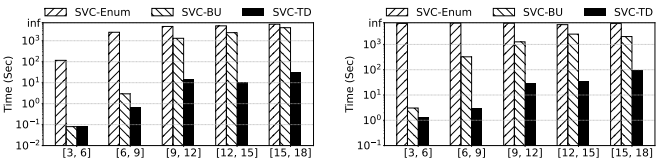
Thus, no better solution is lost by returning after the update. No construction-failure case is needed, because the size and threshold checks guarantee the construction whenever $p < t$.

Together with complete `Sym-SE` branching and safe pruning, this argument shows that an iteration leaves no improving community with $\phi(g) < \theta$, establishing the invariant for the next iteration. At termination, $\theta + k^* > h$, and every better community would satisfy $\phi(g) \le h - k^* - 1 < \theta - 1$, again contradicting the invariant. Hence the returned community is optimal whenever one exists. □

APPENDIX B
ADDITIONAL EXPERIMENTS

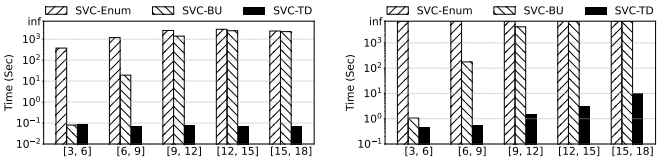
A. Running Time on Sparse and Dense Regions

We further evaluate the algorithms on sparse and dense regions of the two representative datasets, as shown in Figures 9 and 10. Specifically, sparse regions are formed by the 10% vertices with the smallest core numbers among vertices whose core numbers are at least 5, while dense regions are formed by the 10% vertices with the largest core numbers. We have the following observations. First, `SVC-TD` achieves the lowest running time in all tested cases. Second, `SVC-TD` is faster on dense regions than on sparse regions. This is because dense regions usually contain communities with larger optimal `VC` values, allowing `SVC-TD` to identify high-`VC` communities earlier under tighter $\phi(\cdot)$ thresholds. Once a better community is found, the increased k^* further strengthens subsequent pruning. Third, `SVC-Enum` and `SVC-BU` do not show a clear difference between sparse and dense regions, and both are much more time-consuming than `SVC-TD`. Although all three algorithms use an improved connectivity lower bound to prune the search, only `SVC-TD` shows a clear runtime advantage on dense regions in these experiments.



(a) DBLP (b) YouTube

Fig. 9. Running time on sparse regions



(a) DBLP (b) YouTube

Fig. 10. Running time on dense regions